

WORKSHOP • PART 1 OF 2

Model Context Protocol (MCP)

A hands-on session on MCP fundamentals — concepts, architecture, capability design, and how to build and register one on our platform.

Senior Architects Track • Followed by: Agentic AI with LangChain & LangGraph

Concepts • live build • your own MCP, registered by the end of this session.



AGENDA

Agenda — the next ~1h30

A couple of short breathers are built in along the way.

	Warm-up — opinion spectrum & where we stand today	10 min
	What is MCP — concepts, architecture, lifecycle	15 min
	Capability design — Tools, Resources, Prompts	20 min
	Platform — roadmap, architecture, deployment models	15 min
	Registering an MCP — registry, naming, guidelines	10 min
	Integration, security & observability essentials	15 min
	Frameworks — Python (FastMCP) vs Java (Spring AI)	5 min
	Hands-on — build & register your own MCP	Live demo

ICEBREAKER

The Opinion Spectrum

Agree or disagree? Move to a side of the room — or just call it out. No wrong answers, only opinions.



The biggest bottleneck for AI agent adoption in enterprises today is the difficulty of accessing real-time data.



A standardized protocol like MCP is preferable to creating custom API integrations for every new tool.



MCP acts like a universal adapter (similar to USB-C), letting different AI applications connect easily to data sources and tools.



A new technology will emerge and replace MCP in the agentic world.



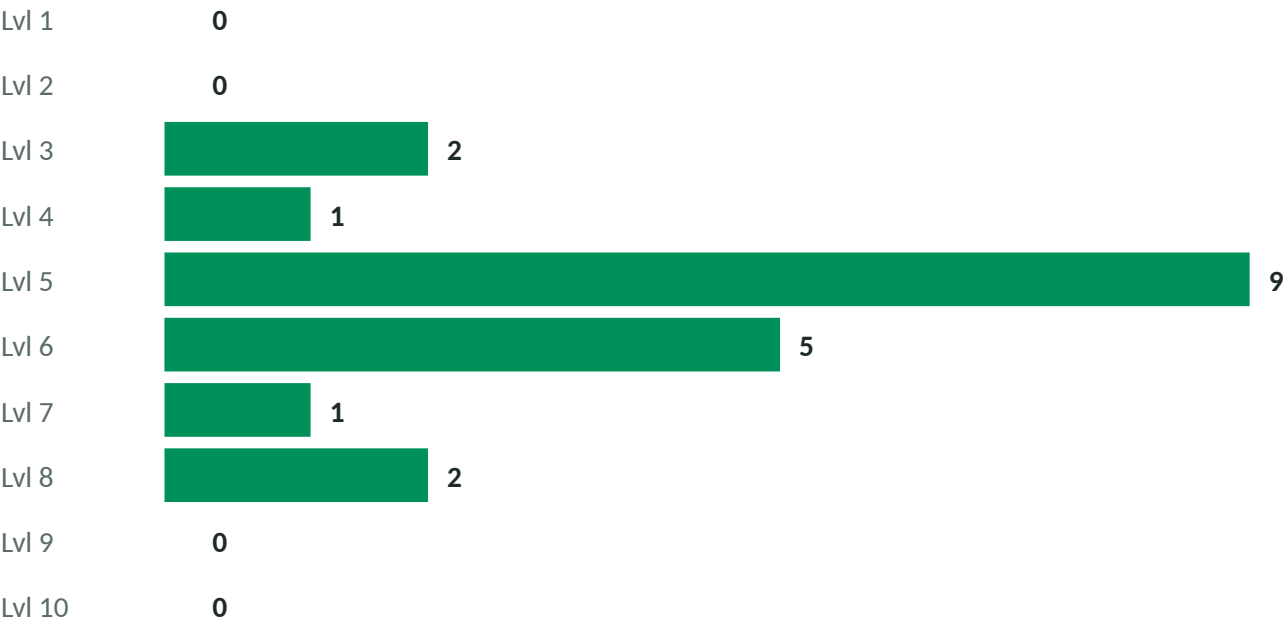
In the future, knowing how to build MCP servers will be as fundamental a skill for a developer as knowing how to build a REST API.



Population today

On a scale of 1 to 10, how would you rate your knowledge of MCP?

1 = "Never heard of MCP."
5 = "Know the theory but have never built one."
10 = "Built multiple MCPs and mentor others."



5.40

sample average
Model Context Protocol (MCP)

Who's in the room — roles

Developer	Technical Architect	
Java Developer	App. Manager	AI Catalyst
Project Manager	Senior	Business Analyst
Architect / AI	Manager	Team Lead

(illustrative sample — today's live poll will differ)

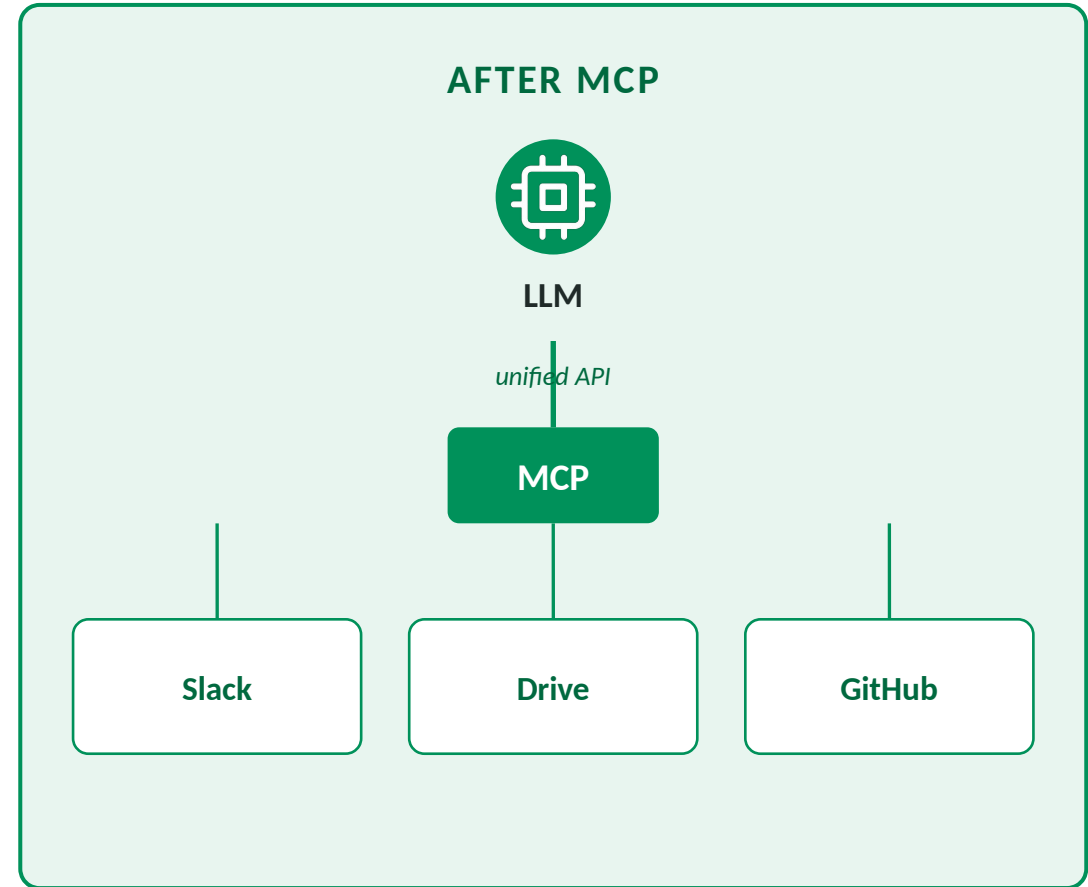
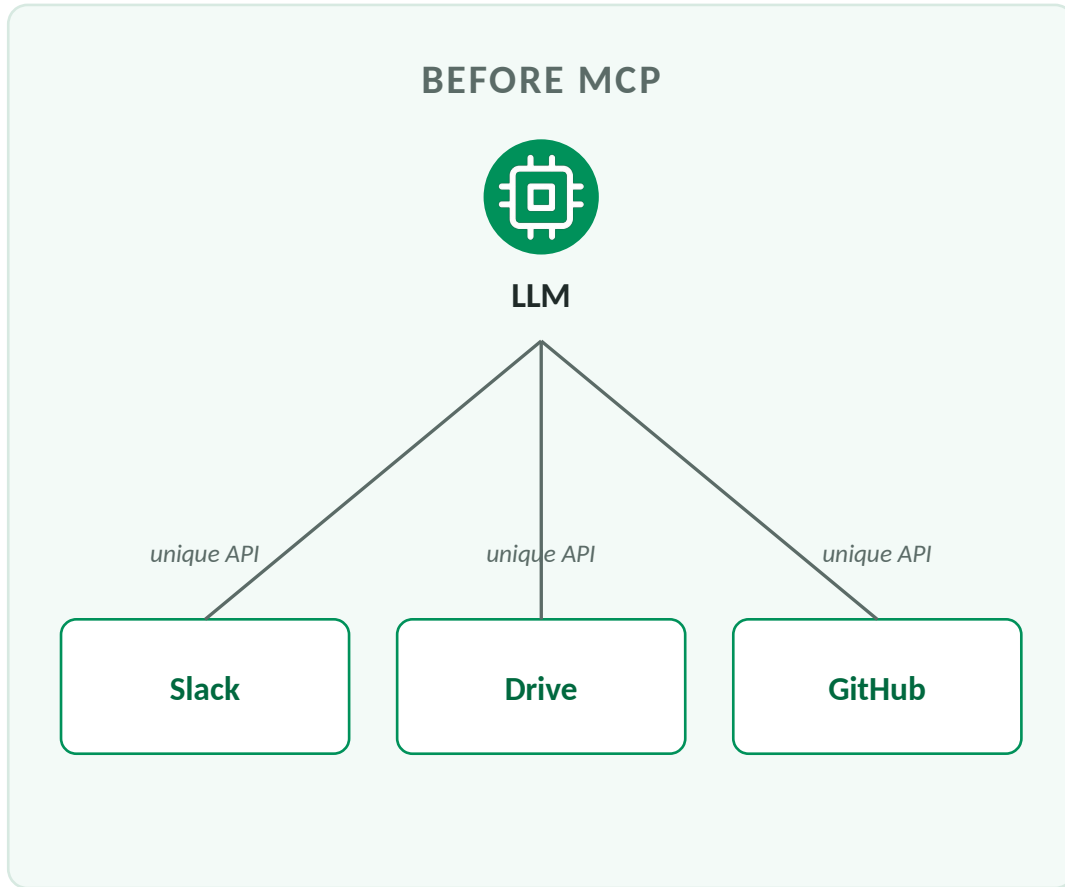


What is Model Context Protocol (MCP)

Model	Context	Protocol
The LLM at the heart of your application — GPT, Claude, Qwen or whatever. A powerful reasoning engine but limited by what it was trained on and how much context it can hold.	The extra information your model needs to do its job — documents, search results, user preferences, recent history. Context extends the model's capabilities beyond its training set.	A standardized way of communicating between components. Think of it as a common language that lets your model interact with tools and data sources in a predictable way.

MCP is an open protocol that standardizes how AI applications interact with external systems such as tools, knowledge sources, and prompting templates — a unified interface between AI clients, LLMs and capability providers.

What is MCP — before and after



Same tools, one integration point — the model talks to MCP once; MCP talks to every tool.

What is MCP — client-server architecture

MCP uses a client-server architecture built on JSON-RPC 2.0. A single host application (Claude Desktop, Claude Code, Cursor, RooCode etc.) creates multiple isolated MCP client sessions, each maintaining a stateful JSON-RPC channel with its own MCP server.

Component	Role	Examples
MCP Host	AI application providing LLM integration	Claude Desktop, Claude Code, Cursor, Cline, RooCode
MCP Client	Connector created by the host, managing 1:1 sessions with servers	Internal process (not exposed to users)
MCP Server	Service providing external context and capabilities	GitLab, Confluence, Jira, PostgreSQL MCP servers

JSON-RPC 2.0, in practice

User asks: "Give me the information for JIRA Ticket n°30043"

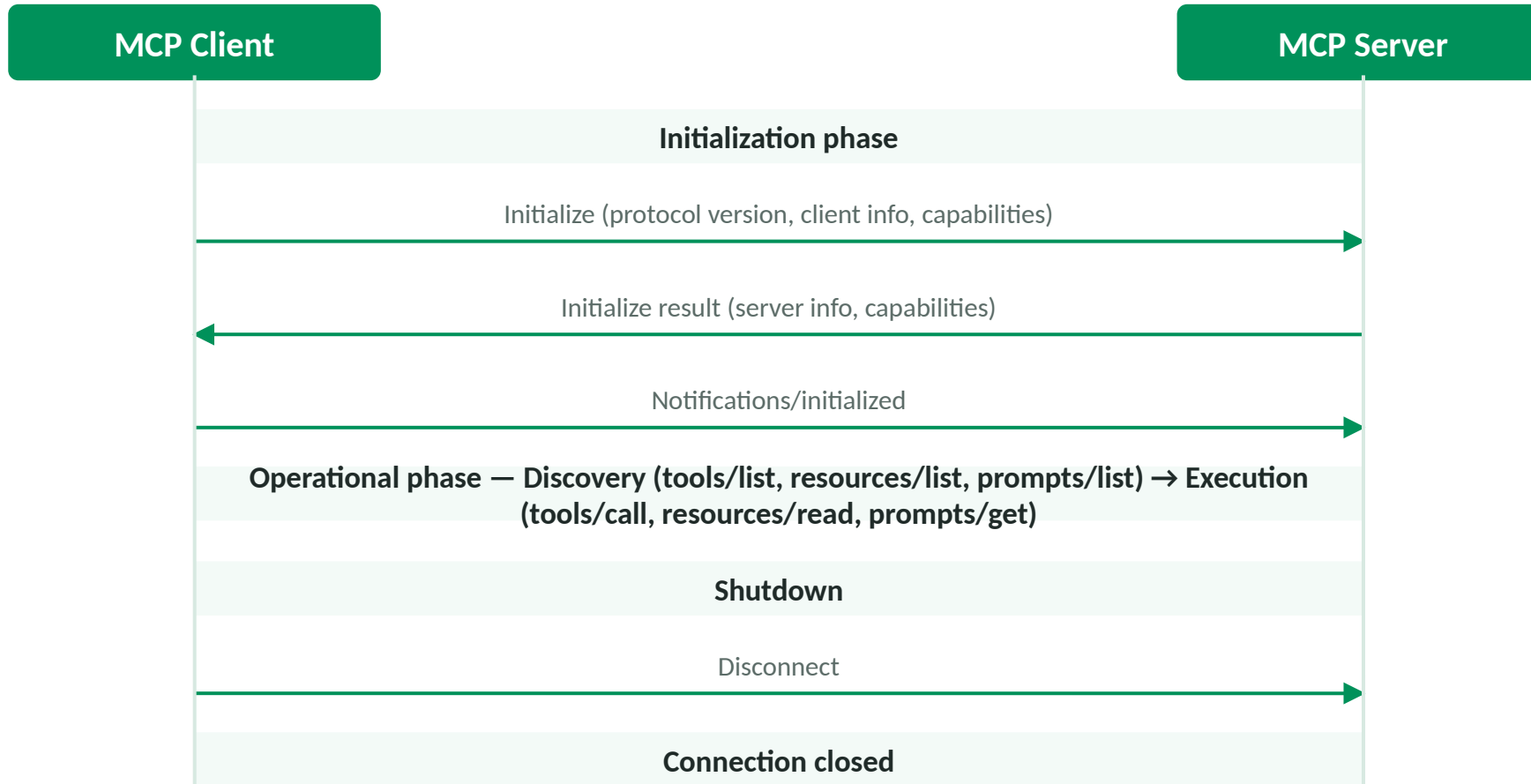
```
{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "params": {
    "name": "get_jira_issue",
    "arguments": { "issue_id": "30043" }
  },
  "id": 2
}
```

Input to MCP Server

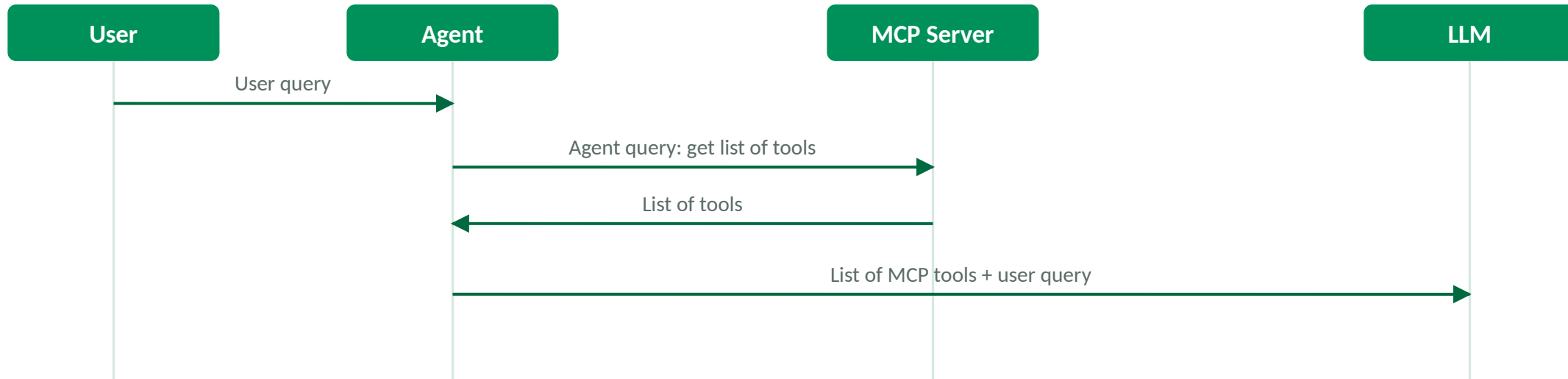
```
{
  "jsonrpc": "2.0",
  "result": {
    "content": [
      {
        "type": "text",
        "text": "Issue JIRA-30043: [Bug]
              500 Error on Login Page.
              Status: In Progress."
      }
    ]
  },
  "id": 2
}
```

Output from MCP Server

MCP lifecycle



MCP usage lifecycle — tools in practice



For each new session, the agent loads MCP tools into its context. Unlike a human handyman who knows their tools by heart, a large tool list can consume over 20% of the context window before the agent even starts working — keep tool counts per server lean (rule of thumb: under 60).



Core principles

Capabilities (not implementations)

Must: describe what the capability does in domain language.

Must: expose stable, contract-based boundaries.

Avoid: leaking internal implementation (API contracts, internal IDs, workflow steps).

One capability (one explicit intent)

Must: small, single-purpose capabilities with clear impacts and limits (smaller blast radius, easier authorization).

Avoid: "god capabilities" that do many things (hard to govern, test and evolve).

Capabilities exposed by MCP servers

Tools

- › Executable actions — e.g. create a ticket, query an API, trigger a workflow.

Resources

- › Provide contextual data or knowledge — e.g. documentation, database records.

Prompts

- › Reusable LLM prompt templates.



Designing tools

Tools

Single Intent: each tool performs one bounded action with a predictable outcome.

Interfaces are contracts: schema-first inputs/outputs + standard error structure.

Design for consistency: same input and conditions, same result (idempotent where possible).

In practice

Atomic: perform a single, self-contained action — no tool-to-tool calls or MCP server chaining.

Actions: Create, Update, Add, Get.

Examples: `send_email()`, `fetch_sales_data()`, `create_meeting()`.

Agents decide, tools execute — tools should be bounded, observable, and policy-compliant (timeouts, limits, audit).



Designing resources

Resources

Context-first: resources expose contextual data that supports agent reasoning.

Contract-first: structured payloads with clear schemas and stable identifiers/URIs (e.g. users://{user_id}/profile).

In practice

Read-only by design: resources never trigger actions or side effects.

Scoped ownership: resources are domain-scoped by default.

Operable access: pagination/filtering, size limits and cache. Examples: policy documents, file contents.

Resources provide the "knowing", tools provide the "doing".



Designing prompts

Prompts

Guidance, not logic: prompts shape model behaviour. Decisions and execution stay in agents/tools.

Reusable + governed: prompts are shared assets with ownership, review, and change logs.

In practice

Explicit and scoped: each prompt has one purpose (domain/task/style).

Safe by default: no secrets or sensitive data embedded.

Tested and versioned: evaluate, version and roll back fast if needed.

Prompts are governed production assets and must follow the same SDLC, security review and audit controls as code.



When to use a ...

Tool	Resource	Prompt
Use when: the AI needs to take action or the task requires real-time data. Sending an email? Tool. Fetching live weather? Tool. Writing to a database? Tool. The test: if there's a side effect (something changes in the world), it's a Tool.	Use when: the AI needs context or background information to answer accurately. Company policy documents? Resource. Product catalog? Resource. Historical sales data? Resource. The test: if it's read-only information that grounds the AI's response, it's a Resource.	Use when: you want consistent AI behavior across sessions. "Always write in our brand voice." "Always review code for these five specific issues." "Always summarize in exactly 5 bullets." The test: if you're establishing a pattern you'll repeat many times, make it a Prompt.

If the model needs to ask for it by argument, it's a tool. If the user (via the client UI) wants to attach it to context, it's a resource.



Link to the right capability

Give me the current price of Widget-X and, if it's below \$120, place an order for 12 units for my account #4567

→ **Tools**

What files were changed in merge-request !42 of GitLab project my-group/my-project?

→ **Tools**

Discuss: what would make either of these a Resource or Prompt instead, if you changed one word?



Quick stretch — two minutes

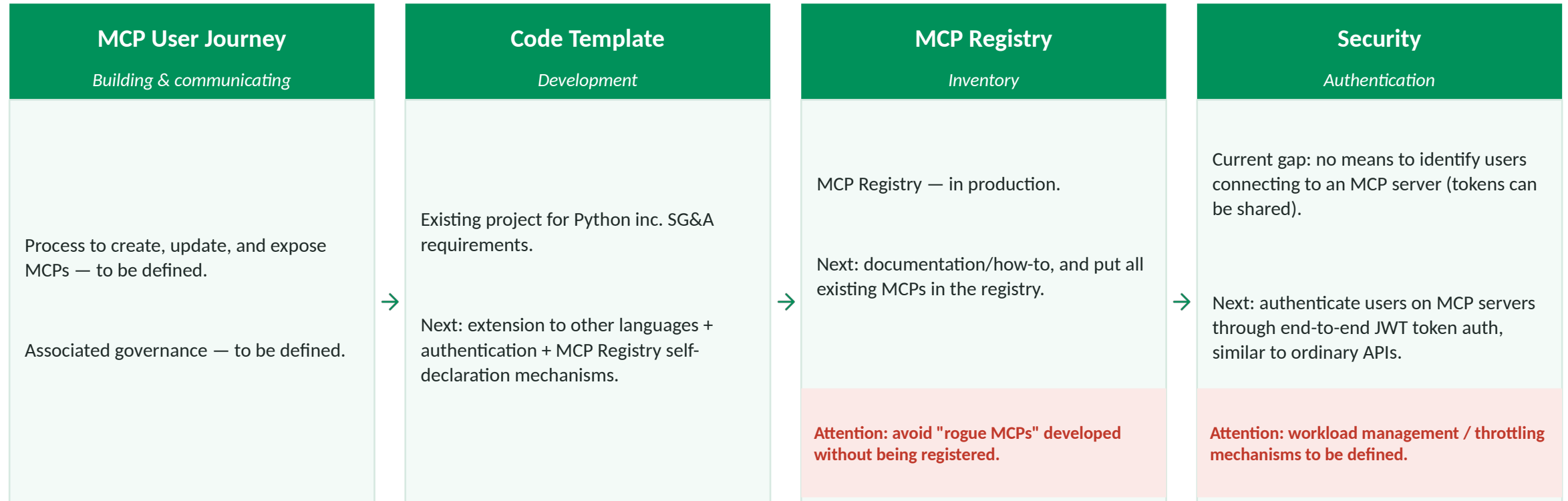
“Which capability — Tool, Resource, or Prompt — does your team most often get wrong today?”



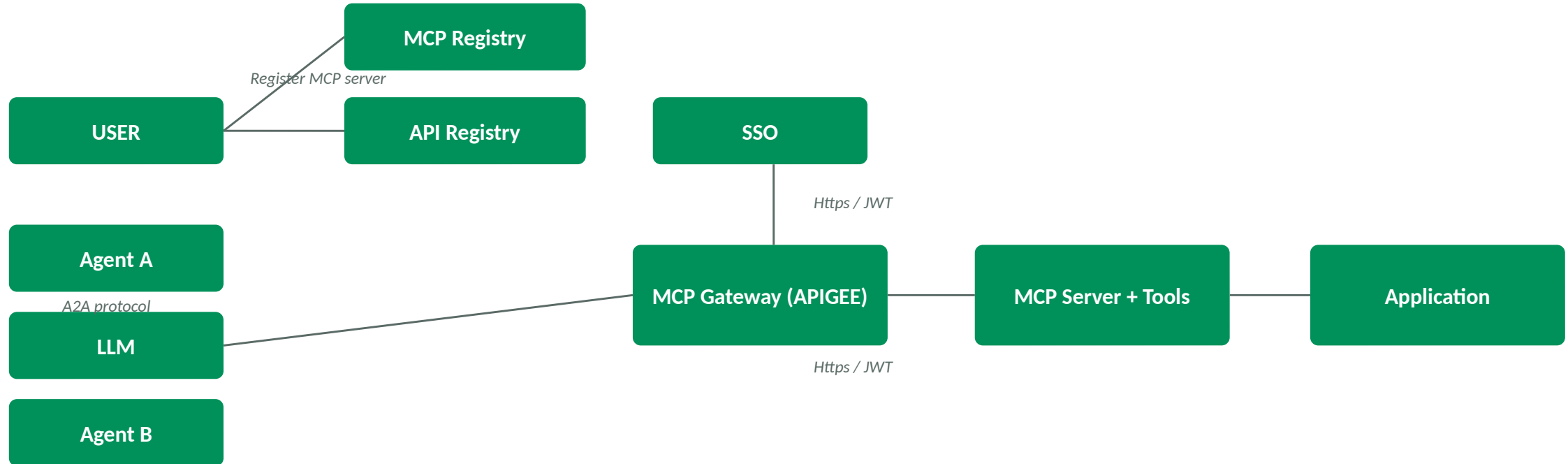
Stretch, top up the coffee, then a few quick answers before we go platform-side.



MCP framework roadmap



Architecture guidelines



Gateway & MCP controls

An MCP Server is registered with its tools in the API Registry (as an MCP), and supports standard OAuth authorization via APIGEE: (1) the MCP server verifies agent entitlements, (2) provides the list of tools the agent is entitled to (least privilege), (3) tools are wrapped with guardrail + entitlement validation hooks, (4) upon validation, the process is triggered using on-behalf-of JWT tokens.



Architecture guidelines — service boundaries

MCP Server

Each MCP Server is the service provider for exactly one application domain.

Exposes atomic tools/resources for a single application domain.

MCP Servers do not call other MCP Servers.

Interaction & Boundaries

MCP clients interact only via MCP server tools or resources.

MCP Servers should be stateless with respect to agent or user conversation context.

Stateful workflows and memory belong in the agent or orchestrator layer.

Ownership & Isolation

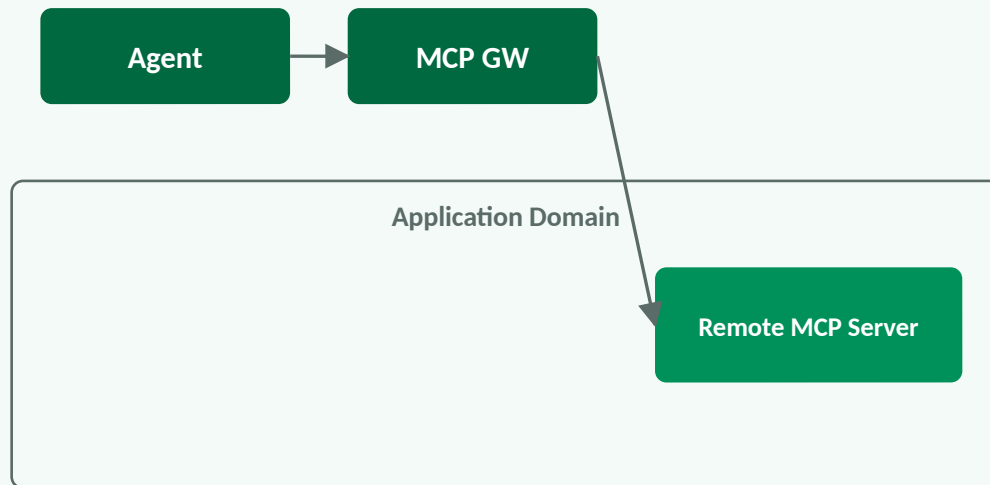
Each MCP Server must have a single owning team responsible for implementation, security, operations, and support.

MCP Servers are discoverable only via the MCP Registry.

Deployment models of MCP server

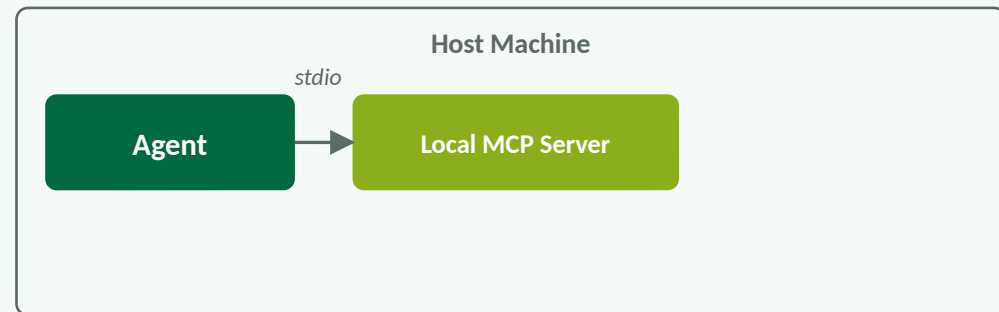
Remote (Streamable HTTP)

Default for shared, reusable capabilities across multiple agents.
Centralized governance of tools, resources, prompts, and contracts.



Local (Stdio)

Preferable for development, prototyping and testing. Limit to local-only available capabilities (e.g. filesystem access). Scenarios requiring extremely low latency.



The choice between local and remote is driven by reuse, governance, operational needs, and risk/security requirements.



Token Management

Cover at minimum:

Issuance & rotation

Scoping per capability

Revocation



Content to be added by the organization

How tokens are issued, scoped, refreshed, and revoked for MCP clients, gateways, and servers in our environment.



MCP Registry — how to register

Register at (per environment):

STG – `registry.llm.staging.<org>/mcp`

PRD – `registry.llm.<org>/mcp`



1. Add a new server

From the registry's MCP Servers view, click the "+" to register a new server.



2. Fill in the registration details

Complete the required fields (see next slide) — type, title, name, tools, headers, and more.



3. Get approval from the platform team

Your MCP will be in "Draft" mode pending review. Contact the platform team if you need help with approval.

MCP Registry — required information (1/2)

Field	Type	Description	Validation / Notes
MCP Type	SSE or streamable-http	Determines the communication protocol the server implements.	Must be one of the two supported values.
Title	Short string (50 chars)	Human-readable label that appears in the drop-down/selector.	Must be unique across all registered MCPs.
Name	Short string (50 chars)	Prefix added to the MCP server’s tools; discoverable name by the LLM.	Must be unique and not too complex.
Description	Short string (150 chars)	One-sentence summary that helps users understand the server’s purpose.	Avoid internal jargon.
Icon URL	URL / data-URI	Graphic shown next to the name in the selector and detail card.	Must be a reachable HTTPS URL or Base64 SVG data URI.

MCP Registry — required information (2/2)

Field	Type	Description	Validation / Notes
Tools	List of tools	Choose the tools that the LLM can call for this MCP.	Whitelist (only these allowed) or blacklist (all except these).
Custom Headers	List of objects	Extra HTTP headers the client must send (e.g. an auth token).	is_secret: true hides the value in the UI and stores it encrypted.
Use case URL	URL	Link to the use case entry in the internal inventory (governance).	Required for validation.
Support email	Email	Team to answer any support queries.	Required.
User guide URL	URL	Link to documentation to help use the MCP in question.	Optional but recommended.

Guidelines — naming conventions

MCP Servers	› mcp-<applicationDomain> or mcp-<applicationDomain>-<scope>. Examples: mcp-jira, mcp-identity-core, mcp-datawarehouse.
Tools	› <verb>_<object> (snake_case). Examples: create_ticket, update_issue_status.
Resources	› <noun> or <noun>_<qualifier> (no verbs). Examples: project_metadata, user_roles, inventory_snapshot.
Prompts	› <domain>_<intent>_prompt. Examples: jira_summary_prompt, incident_analysis_prompt.
Version	› Managed via registry/metadata, not embedded in names. Preferred: create_ticket @ v1.2.0. Avoid: create_ticket_v2, mcp-jira-v1.



Integration essentials

Integration

Must: wrap existing or develop new APIs instead of accessing databases or queues directly — reuse the business logic those APIs expose.

Must: register every remote MCP server in the MCP Registry.

Must: expose every remote MCP server through an MCP Gateway.

Contracts & schemas (JSON-RPC)

Must: schema-first contracts per capability (params/result/error).

Must: explicit error taxonomy; validate on the server (optionally on the client for fast failure).

Must: version contracts in metadata (contractVersion, schemaId), not in names.

Avoid: free-text outputs for machine actions — return structured results.

Reliability

Must: timeouts for every call + circuit breakers + bounded retries, as API logic.

Must: retries only when safe (read-only by default; writes require an idempotency strategy).

Must: canonical errors with retrievable/non-retrievable classification, health/readiness checks, and logs including capability ID + version + correlation ID.

Sampling vs. elicitation

Sampling (requests to the model)

A way for servers to request LLM generations from language models via clients — include human-in-the-loop for sampling requests.

Without sampling, an MCP server has two limited choices: return raw data and hope the client model knows what to do, or call its own model provider directly (raises cost, privacy, and key-management issues).

Elicitation (requests to the user)

Allows servers to dynamically request specific information from users during an interaction.

A structured mechanism to gather additional input only when needed, keeping the AI's context clear and precise.

Whenever the AI lacks sufficient context to proceed, the MCP server can trigger an elicitation request to collect the missing piece of data from the user.

Async / long-running workflows: use a message bus for async work and expose as tools via MCP — `submitJob`, `getJobStatus`, `getJobResult`, `cancelJob`.

Elicitation — a worked example

e.g. "Check my last three out-for-delivery orders and email me the details."



It first calls `getOrdersByStatus(status="OUT_FOR_DELIVERY", limit=3)`



It then prepares a summary via a prompt and sampling



Now it needs to send an email using the `sendEmail` tool — but it encounters multiple email addresses associated with the user. Without elicitation, the AI might guess which email to use and send data to the wrong address.



But with elicitation, the server initiates a request, prompting the client to display a simple input form: "Which email should I use to send your summary?"



Security — network, auth & secrets

Network	Auth & Authz	Secret Management
Must: TLS 1.2+ for all remote MCP endpoints (HTTPS only). Must: controlled ingress (MCP Server Gateway) with inbound allowlists. Avoid: unrestricted egress — allowlist outbound destinations per capability and log all outbound access.	Must: authenticate every remote request (MCP supports OAuth by default). Must: deny-by-default + least privilege (CBAC), with short-lived, audience-bound tokens mapped to allowed capabilities. Avoid: token passthrough to downstream systems — use the on-behalf-of pattern instead.	Must: secrets only in an approved secrets manager. Must: automatic secret rotation. Avoid: logging tokens/secrets. Never embed secrets in prompts or return them in tool outputs.

Treat an MCP with the same rigor and discipline used for an API.

Security — tools & data

Tools

Must: capability-based access control (CBAC) — agents can only invoke tools/resources/prompts they're authorized for.

Must: tool metadata and prompts should be scanned in CI/CD for injection patterns (e.g. delete, exec, curl).

Avoid: "super tools" with broad permissions (e.g. one tool that can read + send + delete emails).

Data

Must: tag resources with data classification and enforce least-privilege access, including context shared through resources/tools.

Must: encrypt data in transit and at rest; restrict export/sharing to explicitly approved capabilities.

Avoid: logging sensitive data — use masking/redaction/encryption and time-bound retention where logging is required.

Treat an MCP with the same rigor and discipline used for an API.



Observability

Logs

Must: all errors logged.

Must: structured logs with consistent fields.

Avoid: logging secrets/credentials or sensitive payloads.

Traces & Correlation

Must: propagate trace IDs end-to-end (host → MCP client → MCP server → downstream).

Avoid: "generic error" or null responses. Return canonical error codes + messages.

Metrics & Audit

Must: audit events for side effects — WHO did WHAT and WHEN (plus target resource + outcome).

Must: core metrics per capability: request rate, error rate, latency, timeouts, retries, etc.

Send logs/metrics/traces to the enterprise observability stack.



Frameworks — Python vs Java

Python — FastMCP (v1, v2, v3)

Most widely adopted, developer-friendly MCP framework.

High-level abstraction: decorator-based registration, automatic schema generation.

Production-ready (~70% of MCP servers in the wild), with multiple transport options (stdio, HTTP/SSE) and four built-in auth strategies.

Java — Spring AI

Dominant AI framework in the Java ecosystem.

Extends the Java SDK with Spring Boot ecosystem integration; easier configuration, one-line starter import.

Annotations-driven (MCP Annotations), with MCP Security on OAuth 2.0 and AI key-based support.

Main advantages of FastMCP and Spring AI: user-friendly, compatible, production-ready, easy to set up, secure.

ICEBREAKER



One for the break

“If your MCP server had a support ticket filed against it in six months, what would it most likely say?”



A few quick answers — then a proper break before we build.



Break

30 minutes — back for hands-on MCP building.



Build & register your own MCP

Required — minimum bar

- ✓ One tool with a clear single intent, following the naming convention
- ✓ A schema-first contract (params/result/error)
- ✓ Registered in the staging MCP Registry, in Draft mode

Stretch — if you finish early

- ★ Add a resource exposing read-only context for the same domain
- ★ Add a reusable prompt template for a common task in that domain

Pick a domain:

Jira / Tickets

Internal Wiki

HR Directory

Live demo first, then it's your turn — registration stays in Draft mode, so nothing goes live without review.